# Judge vs. React: dissociating perception from timing in VLM reflex control

Josh Simnitt
jsimnitt@gmail.com

2026

**Abstract**

A vision-language-model (VLM) agent that must time one action in a first-person scene can fail two ways that aggregate game scores cannot pull apart: misjudge the scene, or judge correctly and act too late. We show the conflation concretely. A closed-loop Sonnet 4.6 agent, judging every frame of a deterministic first-person pit-jump task itself, clears **41/72** episodes in clear weather and **41/72** under fog, for opposite reasons: 36 correct in-window decisions plus 5 early misjudgements carried into the window by the control delay, versus 19 plus 22. Fog spread the judge's commit point from one tick to five and raised early commits from 9 to 35; the delay rescued most of the damage. All 144 episodes obey one rule: the episode clears iff the commit position plus the distance travelled during the delay lands in the window. Same score, different agents. A second, cheaper judge (Haiku 4.5) through the same grid commits two ticks later and its cliff moves by exactly two ticks ($r^\star = 0.96$); its early commits are far-field first-passage errors no delay can rescue, and it clears 26/72, as the same equation predicts from its commit distribution. Removing the delay knob and sweeping the clock instead, so that the judge's own latency is the delay, the same judge clears 0/30 episodes until $r$ falls below 1/3 (two consecutive JUMPs must fit the window) and 18/18 once the clock is slow enough; in between, 10 of 11 failures are a single event, a 15–30 s deliberation on the frame just before the window, where the model answers in 2.6 s everywhere else. At low effort the stall is gone and the same frames are misjudged instead (54% false JUMP), so the judge commits early in 58/72 episodes and clears 8/72. In real time the budget is the longest latency on the approach, the longest one falls on the hardest frame, and effort trades that stall for a first-passage failure on the same frames.

The apparatus reduces an episode to one decision, labels it in- or out-of-window from the sim's own dynamics, and injects a control delay $D$ so that lateness varies independently of judgment. In the coordinate $r = vD/W$ a leading-edge policy clears iff $r < 1$ by construction; oracle sweeps ($r^\star = 1.04$ on the pit, 0.98 on an aiming task) confirm that the apparatus reproduces this and serve as calibration, not as a law. Swept through that boundary, the same Sonnet agent clears **32/32** in-window decisions below $r \approx 1$ and **0/26** above it ($r^\star = 1.05$) with nothing varied but the delay. Its commit point is nearly deterministic (63/72 within one tick), so the transition is a step, not a psychometric curve, and because it is asked on every one of ∼90 approach frames, a 1–2% per-frame false-JUMP rate fires almost surely somewhere unless a commit rule suppresses it.

On a balanced 100-frame perception battery through the API, no model judges the depth cue unprompted; every hosted model does once the prompt names it (91–99%), fog then costs the smallest most (Haiku 97% to 68%), and a local 7B model answers constantly either way: perception of this scene is a property of the prompt. Five real models' decision latencies all exceed the task's 257 ms window ($r \approx 4$–$47$), so none can act in time in real time, yet the same decisions succeed paused. "VLMs are too slow to play games" is partly an artifact of the time

regime they are benchmarked in, and an aggregate score cannot tell a good judge from a lucky one.

# 1 Introduction

When a vision-language model plays a real-time game and walks into a pit, the post-mortem is ambiguous. Did it fail to *see* that a jump was needed (perception), or did it see correctly and *act too late* (timing)? Aggregate game benchmarks — score, win rate, levels cleared — fold these two failures into one number, so "the VLM is bad at this game" can mean either, and usually means some unknown mix. The two have opposite fixes: a perception failure wants a better model; a timing failure wants a slower clock. Conflating them is why "VLMs are too slow to play games" is repeated as though it were a single fact.

Our move is isolation. We reduce an episode to *one* correct decision — a first-person pit the player auto-walks toward, where the only choice is *when* to jump — so that on any given run exactly one of the two failures can occur, and we can say which. The apparatus is deterministic and headless, and it injects a control delay $D$ (a decision made at tick $t$ is applied at $t+D$), which lets us vary *lateness* completely independently of *judgment quality*: a perfect-judgment oracle still fails if its action lands too late.

The headline result (§4.4) is a closed-loop Sonnet 4.6 agent, judging every frame itself, swept through the boundary by injected delay: it clears 32/32 in-window decisions below $r \approx 1$ and 0/26 above it, and its only clears past the boundary are *delay-rescued misjudgments*. Around it:

- **Calibration (§4.1–4.3).** For a policy that fires at the window's leading edge, clearing iff $r < 1$ is arithmetic, not a discovery. Oracle sweeps on the pit ($r^\star = 1.04$ on a 1-tick delay grid that brackets the edge at every width) and on a rotational aiming task ($r^\star = 0.98$) check that the sim's dynamics, the in-window label, and $r$ reproduce it without leaking tolerance, and the oracle's same-decision flip previews the real-model result on a perfect judge.
- **Two things the real judge revealed (§4.5, §4.6).** A per-frame judge is a first-passage process: a false-JUMP rate that rounds to zero per frame fires almost surely per approach unless a commit rule suppresses it. And the judge's commit point is nearly deterministic, so its transition is a step rather than a curve.
- **Fog in closed loop (§4.7).** The same judge swept through the boundary under fog: its commit point spreads over five ticks, 35/72 episodes commit early, the in-window boundary does not move, and 22 of the early commits are rescued by delay. Both sweeps score 41/72.
- **A second judge (§4.8).** Haiku 4.5 through the same grid: a step at $r^\star = 0.96$, two ticks earlier than Sonnet's because it commits two ticks later; its early commits are far-field first-passage errors that no delay can rescue; 26/72, as Eq. 1 predicts from its commit distribution.
- **The judge's own latency through the boundary (§4.9).** With no injected delay and the clock rate swept instead, the judge's real-time flip lands at $r = 1/3$, the bound the two-consecutive commit rule implies, and the failures between the regimes are a deliberation stall: 10–32 s calls on the frame just before the window, against 2.6 s everywhere else. At low effort the stall is gone and the same frames are misjudged instead: 58/72 early commits, 8/72 cleared.
- **Perception axis (§4.10).** Through the API, no model judges the depth cue unprompted;

every model does once the prompt names it; fog then costs the smallest model most. An earlier subagent-judged pilot claiming unprompted depth reading is retracted.

- **Latency axis (§4.11).** Five real vision models' decision latencies placed on the same axis: at this sim speed all are past the boundary, hosted frontier models at $r \approx 4$–$6$ and local open VLMs at $r \approx 31$–$47$.
- **Control (Appendix A).** Action-repeat adds polling delay and shifts the transition to lower explicit $r$; it is a cost lever, not a latency fix.

## 2 Related work

Five works are closest; we differentiate from each.

- **OmniGameArena** (Lin et al., 2026, arXiv:2606.09826) — has paused (PDQ) vs. latency-controlled real-time (LCRT) clock modes, but only as **aggregate whole-game** comparisons; no per-decision in-window label, no single-decision flip. We add both, and the $r$ coordinate that makes latency measurements comparable across tasks.
- **VideoGameBench** (Zhang et al., 2025, arXiv:2505.18134) — "Lite" (paused) vs. real-time; names the stale-action mechanism but leaves perception confounded ("it's not just latency"). We resolve exactly that confound by isolating one decision.
- **"Win Fast or Lose Slow"** (Kang et al., NeurIPS 2025, arXiv:2505.19481) — owns the latency–quality tradeoff, but its latency lever is quantization/model-size, which *also* degrades quality, and its ≈200 ms point is an environment action-rate cap, not a task-window threshold. Our sweep varies latency **independently** of the model — the apparatus injects control delay $D$ with the policy fixed.
- **BLINK** (Fu et al., ECCV 2024, arXiv:2404.12390) — canonical "MLLMs fail at depth," but static MCQ. Ours is interactive, parametric, oracle-normalized.
- **Ramstedt & Pal, "Real-Time RL"** (NeurIPS 2019, arXiv:1911.04448) [+ "At Human Speed," arXiv:1810.07286] — the RL parent of the real-time/delay result (monotone degradation for learned policies). We express the same delay cost as $r$ for *pausable VLM agents*, show a real VLM crossing it, and show turn-based deployment stepping back over it.

We explicitly do **not** claim to be first to study the latency–quality tradeoff (Win Fast) nor first to isolate timing from competence in the aggregate (Real-Time Deadlines, arXiv:2601.13206; OmniGameArena). Our claim is the single-decision dissociation on a real model, the compensating-axes observation past the boundary, and the quality-controlled $r$ coordinate that makes the latency measurements interpretable.

## 3 Task and apparatus

A first-person raycaster. The player auto-walks toward a pit; the only decision is *when* to jump. Knobs: pit width (sets the action-window $W$), fog/visibility (perception), approach speed $v$, seed. The sim is deterministic and headless; control delay $D$ (ticks) is **injected** — a decision made at tick $t$ is applied at $t{+}D$ — so $D$ is an independent variable decoupled from wall-clock. Real per-model latency $L$ is measured separately (timed calls) and mapped to the $r$ axis via $L$/tick. An oracle policy (true pit distance) is the perfect-judgment ceiling.

**Ground-truth label.** `in_window` = whether a delay-0 takeoff at the decision position clears, computed from the sim's *own* dynamics (not an analytic approximation), so `in_window` exactly predicts `cleared` at $D$=0. The action window is $W = w_{hi} - w_{lo}$ with $w_{lo}, w_{hi}$ derived by simulating takeoffs; for pit widths beyond the jump reach the window is empty ($W \leq 0$, unclearable) and those configs are excluded from fits (reported, not silently dropped).

$$r = \frac{v \cdot D}{W} \text{ (oracle sweep)} \qquad \text{or} \qquad r = \frac{v \cdot (L/\text{tick})}{W} \text{ (real-model placement).}$$

**What $r$ does and does not tell you.** A policy that commits at the window's leading edge $w_{lo}$ has its delayed takeoff land at $w_{lo} + vD$, which clears iff $vD < W$, i.e. iff $r < 1$. The location of the boundary is therefore fixed by where the policy fires, not discovered by sweeping; a policy that fired at the window's center would place it at $r = 0.5$. $r$ is the coordinate in which that statement is exact and task-independent, which is what makes a measured latency $L$ interpretable against any window. The oracle sweeps in §4.1–4.3 are calibration: they check that the dynamics, the label, and $r$ reproduce the boundary with no leaked tolerance. The question with empirical content — where a *real* judge's commit point falls, and whether its outcome flips at that boundary with nothing else varied — is §4.4.

# 4 Results

## 4.1 Calibration: the oracle boundary

The oracle fires at the first in-window position, so a delayed takeoff lands at $w_{lo} + vD$ and clears iff $r < 1$; the sweep's job is to confirm that the apparatus reproduces this. It does. The default grid (4 pit widths $\times$ 7 delays, $D \in \{0, 4, \ldots, 24\}$) has 28 unique configurations. The 1680 committed rows replicate each 60$\times$ across fog and seed, which the oracle ignores (every replicate is identical), and the 7 configurations at pit width 4 have an empty window ($W \leq 0$, unclearable) and are excluded. On the remaining 21 configurations the pooled logistic fit gives $r^\star = 1.17$, but that number is a grid artifact: the last clear is at $r = 1.04$ and the first failure at $r = 1.30$, both at pit width 2, and 1.17 is the logistic midpoint of a 4-tick gap. The other widths do not straddle the edge within this grid — width 1 ($W = 2.08$) never exceeds $r = 0.81$ and always clears, and width 3 ($W = 0.08$) is already at $r = 3.5$ at the first nonzero delay and always fails.

A 1-tick delay grid ($D \in \{0, \ldots, 40\}$, widths 1–3, one seed, fog 0; 123 configurations) removes the artifact. Each width brackets the edge within a single tick of delay: width 1 between $r = 1.01$ ($D = 30$) and 1.04 ($D = 31$), width 2 between 1.04 ($D = 16$) and 1.10 ($D = 17$), and width 3 between 0.89 ($D = 1$) and 1.77 ($D = 2$), its window being only 1.1 ticks wide. Every width is a clean step (every delay below its edge clears, every delay at or above it fails), and the pooled logistic fit gives $r^\star = 1.04$ (Fig. 1). This is the calibration number used below. The clears just past $r = 1$ are physical: a jump that fires on the same tick can take off from just past the near edge, extending tolerance a fraction of a tick beyond the nominal window. The transition is a step because the oracle is deterministic.

## 4.2 Same-decision flip on the oracle

Restricting to oracle-correct decisions (`in_window` = 1; the oracle's commit position does not depend on $D$), the **identical** decision clears 3/3 unique configurations at $D = 0$ and 0/8 once $r \geq 1.30$. The one in-window decision past $r = 1$ that clears is the $r = 1.04$ same-tick case above.

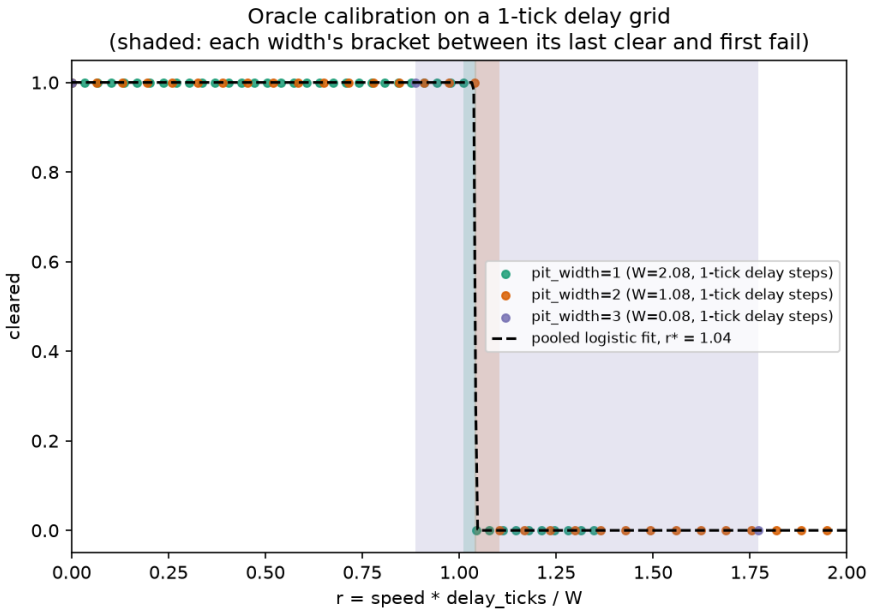



Figure 1: Calibration. Oracle clear/fail vs. $r$ on a 1-tick delay grid; each width's shaded bracket spans its last clear to its first fail, and the pooled logistic fit gives $r^\star = 1.04$.

Only execution timing changed. We do not summarize this as a pooled probability: six of the 9 configurations with $r > 1$ are at pit width 3, where $r$ runs from 3.5 to 21, so a rate over them would describe the delay grid rather than the task. The flip on a real judge, with decisions on both sides of the edge, is §4.4.

### 4.3 The same calibration on a second task

Is the apparatus's behaviour specific to jump-timing? We built a second task in a different modality: a **sweeping-aim** task. The player rotates in place at constant angular speed $\omega$; a target subtends an angular window $[\theta - h, \theta + h]$ (width $2h$); the single decision is *fire*, and a control delay $D$ rotates the aim $\omega D$ further before the shot lands — overshoot if it exceeds the window. By construction the dimensionless cost is $r = \omega D/(2h)$, the exact angular analog of the pit's $vD/W$. The aim episode is pure kinematics and emits the same schema, so the *identical* fit machinery applies.

Because the aim oracle also fires at the leading edge, a hit iff $r < 1$ is again arithmetic; the test is whether an independent implementation of the kinematics and the label reproduces it. It does: across 205 unique configurations at five target widths (4.6°–13.8°; one seed, the aim oracle being deterministic) the pooled logistic fit gives $r^\star = 0.98$, and at every width the last hit lies at $r = 0.87$–$0.96$ and the first miss at exactly $r = 1.00$ (Fig. 4). The offset from the pit's 1.04 is the oracles' firing phase: the pit's takeoff can extend just *past* the near edge (a hair above 1), while the aim fires just *inside* the near edge, forfeiting a sub-tick of tolerance (a hair below 1). Two tasks agreeing on $r \approx 1$ shows that $r$ is the right coordinate for both and that neither implementation leaks tolerance. It is not evidence of an empirical law: a policy that fired at the window's center would place its boundary at $r = 0.5$ in the same apparatus. What the next section asks is where a real judge's commit point actually falls.

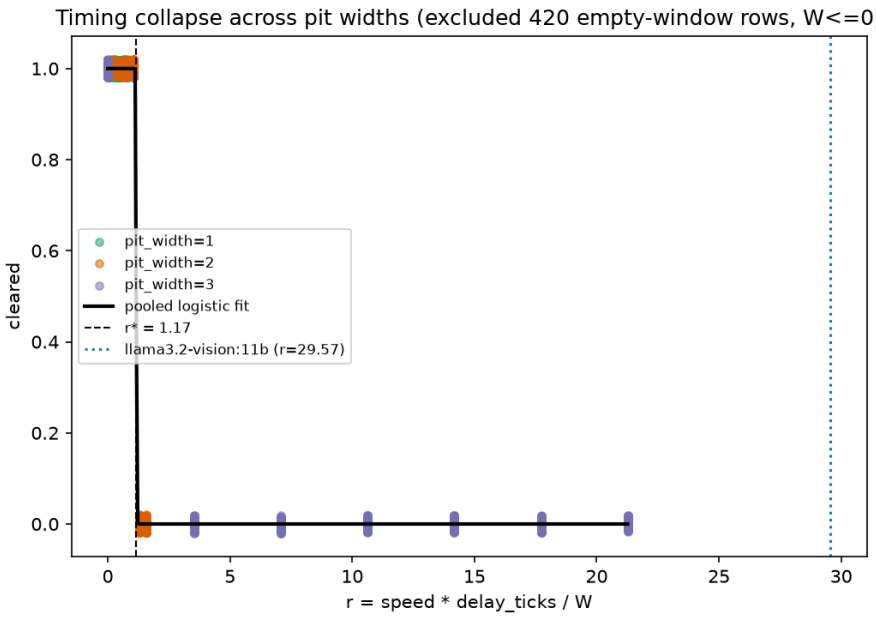



Figure 2: The default oracle grid ($D$ in steps of 4; 21 unique configurations, each replicated 60× across fog/seed in the plotted rows). Its pooled fit of $r^\star = 1.17$ is the midpoint of a 4-tick gap at pit width 2; the 1-tick grid above supersedes it.

### 4.4 A real model through the boundary

The oracle sections calibrate the boundary; here a real model is swept *through* it with its own judgment in the loop. Because the apparatus decouples sim time from wall-clock, the model's (slow, variable) API latency is free, and $r$ is engineered exactly as in §4.1: Sonnet 4.6 judges *every grounded frame* of a closed-loop blocking episode (pit width 2, $v = 0.07$, fog 0, seeds 1–6), its committed jump is applied $D$ ticks later, and sweeping $D \in \{0, \ldots, 31\}$ sweeps $r = vD/W$ from 0 to 2, densely around $r = 1$ (72 episodes; 6,612 vision calls).

**Judge protocol.** The episode prompt used for the latency battery (§4.11) turns out never to elicit JUMP near the window from *any* hosted model: it describes the pit as "the black band across the floor ahead," which stops matching the close-range view — inside the window the pit is a black region filling the bottom of the frame, and the visible checkered floor is the pit's *far* side. The judge used here instead answers a literal perceptual question ("what touches the bottom border of the image: tiles or black?") at high reasoning effort, and commits only on two consecutive JUMP answers; §4.5 explains why that commit rule is forced rather than cosmetic. We validated the final judge on all 108 approach frames × 3 samples (Fig. 6, left): zero JUMPs on the first 79 frames, isolated JUMPs at $x = 8.10$ (1/3) and 8.24 (2/3), and unanimous JUMP from $x = 8.87$ on. This judge was developed against, and validated on, the same renderer, pit width, and fog level the sweep uses; it is a judge tuned to this scene, and we make no claim that it transfers. The bottom-edge cue is nearly ideal in this renderer: the pit's black region first touches the frame's bottom edge at $x = 9.01$, one tick after the window opens ($w_{lo} = 8.92$).

**The boundary holds for a real judge.** Episodes whose decision lands in the window clear **32/32 through** $r = 0.97$, 4/5 at $r = 1.04$, and **0/26 from** $r = 1.10$ **on**; the same logistic fit

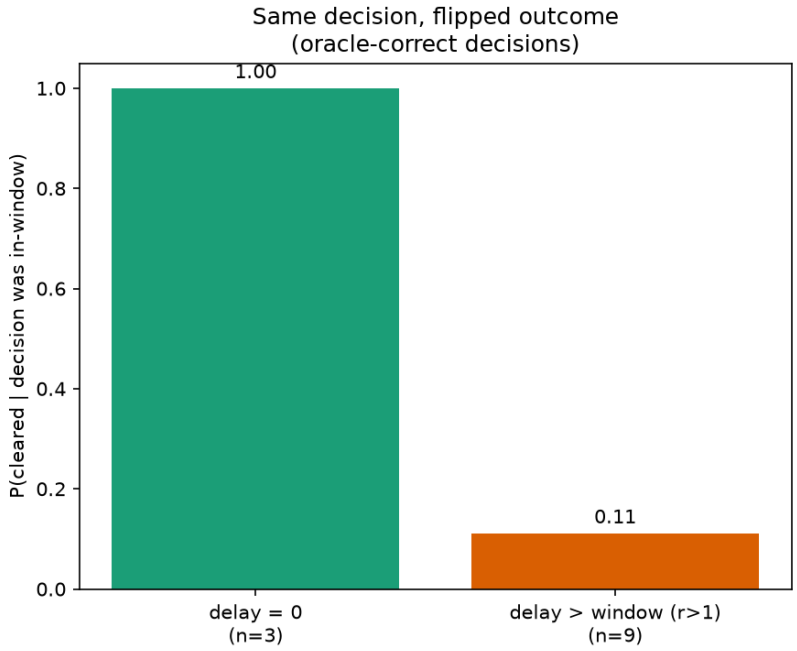



Figure 3: Oracle-correct decisions: 3/3 clear at $D = 0$; of 9 unique configurations past $r = 1$, only the $r = 1.04$ same-tick case clears (0/8 at $r \geq 1.30$).

as §4.1 gives $r^\star = 1.05$, coinciding with the oracle's width-2 bracket $[1.04, 1.10]$ on the 1-tick grid: the real judge's edge sits where the perfect judge's does, same-tick tolerance included (Fig. 5). The transition is nearly as sharp as the oracle's because the model's commit position is almost deterministic (§4.6); given that commit point, the *location* of $r^\star$ follows from the kinematics exactly as in §4.1. What this sweep establishes that the oracle cannot is the dissociation itself on a real judge: the same model, making the same frame-by-frame decisions, clears every in-window decision below the boundary and none above it, with nothing varied but $D$.

**Delay-rescued misjudgments.** 9/72 episodes commit early ($x = 8.10$–$8.24$, out of window — a perception error; these are excluded from the timing fit above). At $D \leq 8$ every such episode fails, landing short. In the band $D = 15$–$24$ ($r = 0.97$–$1.56$), *every* early episode **clears** (5/5): the delay pushes the too-early takeoff into the window. Past $r \approx 1.1$ the *only* surviving episodes are the misjudged ones; by $r = 2.01$ the delay overshoots even the early takeoff and the rescue band closes. The two failure axes do not merely dissociate — on opposite sides of the boundary they *compensate*, which is only visible because the apparatus labels each decision in- or out-of-window.

### 4.5 A frame-by-frame judge is a first-passage process

The commit rule above is not a prompt-engineering detail; the structure of the task forces it. A frame-by-frame controller is consulted on every approach frame, and the episode is decided by the *first* JUMP it commits to, so any per-frame false-positive rate compounds over the approach. With a per-frame false-JUMP rate $p$ over the $N \approx 90$ frames before the window, a single-JUMP commit rule fires somewhere early with probability $1-(1-p)^N$: **60% at** $p = 1\%$ **and 84% at** $p = 2\%$, the rates we measured for earlier prompt versions far from the pit. Under the same independent-error assumption the two-consecutive rule fires with probability 0.9% and 3.4% respectively (Fig. 6, right;

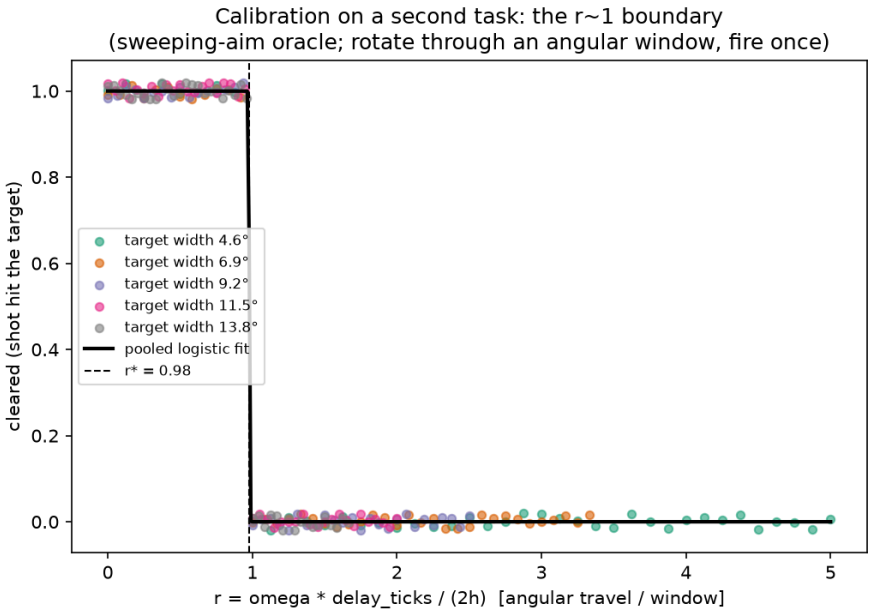



Figure 4: Calibration on a second task. The rotational aiming oracle reproduces the $r \approx 1$ boundary ($r^\star = 0.98$; 205 configurations, 5 target widths).

exact two-state chain). A single-frame accuracy battery — the standard way to evaluate a VLM judge — scores a 1% false-positive rate as 99% accuracy and cannot see this. The final judge's own battery (Fig. 6, left) shows the residual: its only pre-window JUMPs are in the tile before the window ($x = 8.10$–$8.24$), and committing there requires two consecutive misfires, which is what the sweep's 9/72 early episodes are. Its unanimous JUMP from $x = 8.87$, one frame before the window opens, is converted by the commit rule into a commit at 8.94, inside the window. The residual errors are not independent across frames — they cluster where the floor sliver below the pit becomes thin — so the independent-error curves are an idealization; the mechanism is not. Any deployment of a VLM as a per-frame trigger needs a commit rule or an equivalent, and needs to be evaluated per episode, not per frame.

### 4.6 The commit point is nearly deterministic

The sweep's in-window decisions commit at $x = 8.94$ in 51/72 episodes and at $x = 9.01$ in 12 — within one tick of each other across 63 episodes and six seeds, with the judge's own sampling stochasticity (adaptive thinking at high effort) in the loop. We expected a noisy psychometric curve and got a near-step. Two readings. The optimistic one: on a rendered depth cue with a pixel-sharp onset, a frontier VLM's judgment is effectively deterministic at the tick scale, so in a deployment like this one timing failures are failures of latency, not of judgment variance — the clean separation this paper's framing assumes. The cautious one: the judge was tuned to this cue, and its one-tick lead (answering "black" at 8.94, before the region literally reaches the bottom edge at 9.01) shows it is reading a thinning sliver of floor rather than the edge itself. A cue without a sharp onset would presumably spread the commit point, and that spread is the quantity a real-time deployment needs to know, because it sets how much of the window is left after the commit. Measuring it on weaker cues (fog) and weaker judges is the obvious next experiment.

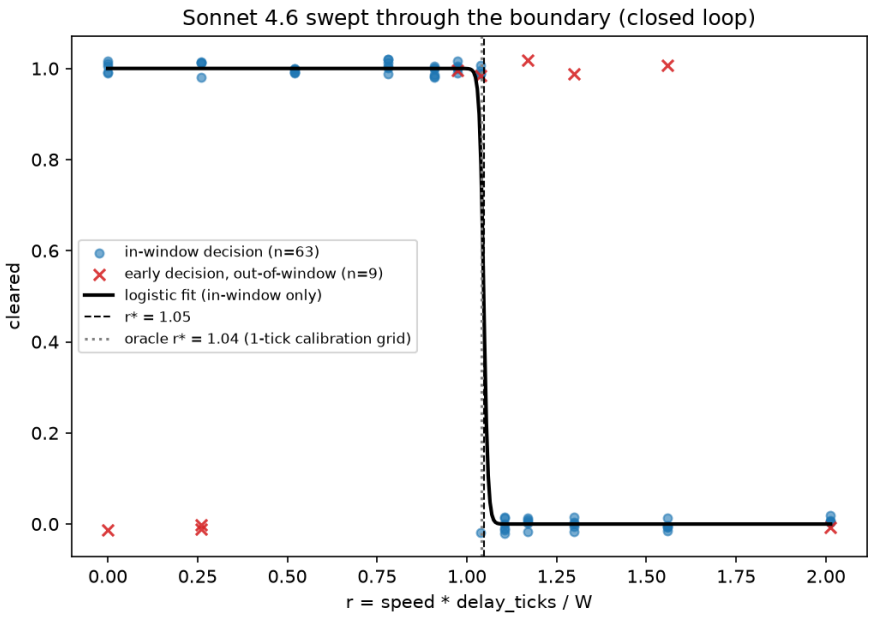



Figure 5: Sonnet 4.6 swept through the boundary in closed loop ($r^\star = 1.05$, in-window decisions). Crosses are out-of-window (early) decisions: they fail below the boundary and are the only survivors past it (delay-rescued perception errors).

### 4.7 The same judge through the boundary under fog

The single-frame battery of §4.10 says fog costs the cue-prompted Sonnet nothing per frame. The §4.4 judge's own battery under fog (108 approach frames × 3 samples at high effort) says it costs something per approach: the judge still fires on every in-window frame, but it also fires early more often — isolated JUMPs at $x = 8.38$, 8.73 and 8.80, a far-field false JUMP at $x = 2.85$ (1/3), and two in-window frames that were not unanimous. We therefore repeated the §4.4 sweep at fog 1: same judge, same prompt, same 12 delays × 6 seeds (72 episodes; 6,669 vision calls), with nothing changed but the renderer's fog (Fig. 7).

**The commit point spreads.** In clear weather 63/72 commits sat at $x = 8.94$ or 9.01, inside the window. Under fog the commits spread over five ticks — 8.80 (11), 8.87 (24), 8.94 (19), 9.01 (17), 9.08 (1) — and **35/72 land before the window opens**, against 9/72 in clear weather. This is the perception failure in closed loop: the same prompt reads the thinning sliver of floor less reliably when fog lowers its contrast, and the judge fires a frame or two early about half the time. It is also the commit-point spread that §4.6 asked for, obtained by weakening the cue rather than the judge.

**The timing boundary does not move.** Among in-window commits the law is as sharp as before: **19/19 clear through** $r = 0.97$ **and 0/18 from** $r = 1.04$ **on**; the logistic fit gives $r^\star = 1.01$ (clear weather: 1.05). The small shift is the commit-offset term of the Discussion: the fog judge's in-window commits at $D = 16$ sat at 9.01 and 9.08 rather than 8.94, so less of the window was left and the same delay overshot. Across both sweeps every one of the 144 episodes obeys a single rule — it clears iff the takeoff lands inside the window extended by the same-tick tolerance, $w_{lo} \le x_{\text{commit}} + vD \le 10.06$ — so the judge's entire contribution to the outcome is where it commits.

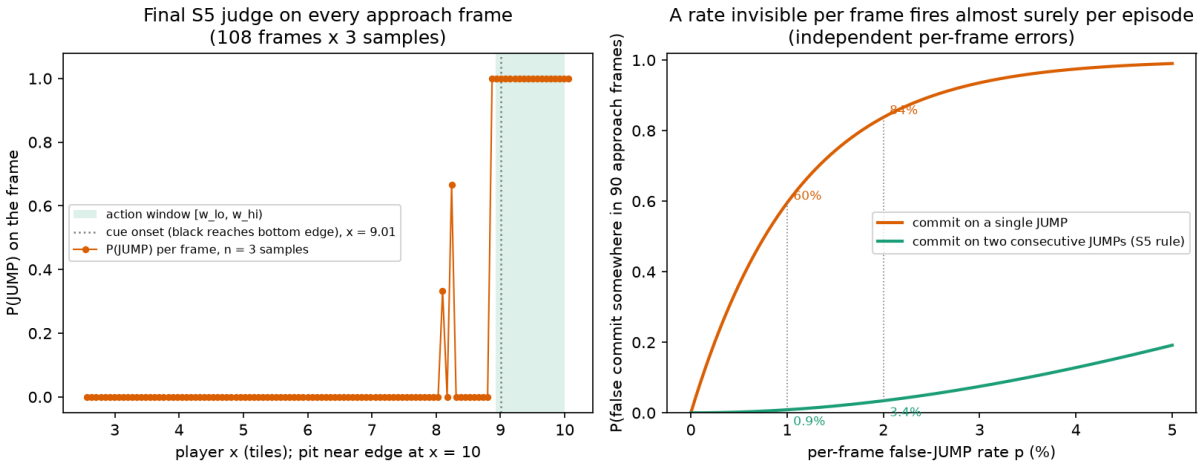



Figure 6: Left: the final judge's per-frame JUMP rate on every approach frame (108 frames × 3 samples) against the window. Right: the probability that a per-frame false-JUMP rate fires at least once over the 90 approach frames, for a single-JUMP commit rule and for the two-consecutive rule, assuming independent per-frame errors.

**The axes compensate at scale.** 22 of the 35 early commits clear: 21 with $4 \le D \le 17$, where the delay carries an 8.80 or 8.87 takeoff into the window, and one 8.80 commit at $D = 18$ that lands exactly on the tolerance edge. Below the boundary the perception errors are invisible (21/21 early commits in that band clear, exactly like the correct decisions), and at $D = 17$ ($r = 1.10$) the only four clears in six episodes are misjudgments. **Both sweeps clear exactly 41/72.** In clear weather that is 36 correct decisions plus 5 rescued errors; under fog it is 19 plus 22. An aggregate success rate cannot tell these apart. Pooling all 72 fog episodes into one logistic fit even produces the smooth psychometric curve a benchmark would report ($r^\star = 1.07$, slope 14, against slope 409 for the in-window fit) — a curve that describes the mixture of correct and early commits, not the judge.

**Why the two scores had to match.** The equality is not a coincidence. Let $F$ be the distribution of the judge's commit position $x_c$; it does not depend on $D$, because the judge sees the same frames up to its commit and the delay only shifts the takeoff. Then

$$P(\text{clear} \mid D) \;=\; F(x_{hi} - vD) - F(w_{lo} - vD), \qquad x_{hi} = w_{hi} + \text{tol}, \tag{1}$$

the probability that the delayed takeoff lands in the window. A sweep sums this over its delay grid. On a uniform grid of spacing $\Delta D$ that covers every delay at which some commit can land, the sum is a Riemann sum of $\int [F(x_{hi} - vD) - F(w_{lo} - vD)]\, dD = (x_{hi} - w_{lo})/v$, so the expected total is $(x_{hi} - w_{lo})/(v\,\Delta D)$ episodes per seed *for any* $F$: a delay-swept aggregate measures the width of the window, not the quality of the judge. Our 12-delay grid is not uniform, so the invariance is approximate there; evaluating Eq. 1 on each sweep's own commit distribution predicts 41.8 clears for clear weather and 41.3 for fog, against 41 and 41 observed, and on a dense one-tick grid both predict $101 \pm 1$ against the limit $6 \times 16.3 = 98$. The same equation explains the shape of the transition: a point-mass $F$ gives a step in $D$, a spread $F$ gives a smooth curve whose slope is the spread, and the pooled fog fit's slope of 14 is the width of the red histogram in Fig. 7, not a property of the timing law.

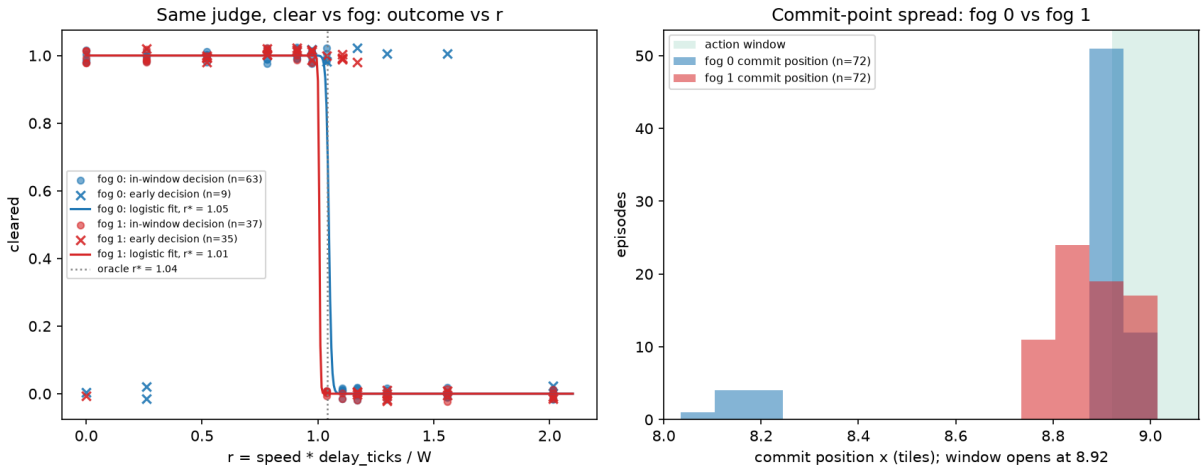



Figure 7: The same judge swept through the boundary in clear weather (blue) and fog (red). Left: outcome vs. $r$; dots are in-window commits with their logistic fits ($r^\star = 1.05$ and 1.01), crosses are early commits, which fail at $D = 0$ and are the only survivors past the boundary. Right: commit-position histograms against the window; fog spreads the commit point over five ticks and puts 35/72 before the window.

### 4.8 A second judge through the boundary

The sweeps above are one judge. To see which of their results belong to the judge and which to the apparatus, we ran the §4.4 grid again with Haiku 4.5 as the judge: same prompt, same two-consecutive commit rule, same 12 delays × 6 seeds, pit width 2, fog 0 (72 episodes, 6,540 calls). Haiku predates adaptive thinking, so it runs with a 2k-token thinking budget; its per-episode mean latency is 2.6 s. On the 108-frame battery it is a worse judge than Sonnet in the ways §4.10 predicts: it fires on every in-window frame except the first ($x = 8.94$, 0/3), and it fires early on scattered far-field frames ($x = 5.02$, 6.49 (2/3), 7.12, 7.40, 8.66) where Sonnet's rate was zero (Fig. 8).

**The step moves by exactly its commit offset.** Haiku commits at $x = 9.08$ in 57/72 episodes, two ticks later than Sonnet's 8.94 (51/72), and its in-window boundary moves by exactly those two ticks: in-window commits clear **24/24 through** $D = 14$ **($r = 0.91$), 1/6 at** $D = 15$ **and 0/31 from** $D = 16$; the logistic fit gives $r^\star = 0.96$ against Sonnet's 1.05. At $D = 14$ all six takeoffs land at $x = 10.06$, on the same-tick tolerance; at $D = 15$ the five from 9.08 land at 10.13 and fall, and the one from 9.01 lands at 10.06 and clears. This is the commit-offset term of the Discussion measured directly, $r^\star = (x_{hi} - x_c)/W$: a judge that commits two ticks later has a cliff two ticks earlier. The transition is a step for Haiku as it was for Sonnet, and every one of its 72 episodes obeys the same rule, cleared iff $w_{lo} \le x_c + vD \le 10.06$.

**Its errors are the first-passage kind.** Haiku commits early in 11/72 episodes and Sonnet in 9/72, but the composition differs. Ten of Haiku's eleven are far-field false commits at $x = 6.49$–$7.47$, two to three tiles before the pit, where no delay on the grid can carry them into the window; one, at 8.73, is rescued at $D = 17$. Sonnet's nine were all in the tile before the window (8.10–8.24) and five were rescued. So Haiku's early commits are the §4.5 failure, a per-frame false-JUMP rate compounding over the approach, which no delay can compensate; Sonnet's were near-edge misjudgments that delay could. Haiku clears **26/72** to Sonnet's 41/72, and Eq. 1 on Haiku's own

commit distribution predicts 27.1: the lower score is the later commit (five clearing delays on the grid instead of six) plus ten unrescuable far-field commits, not a different timing law.

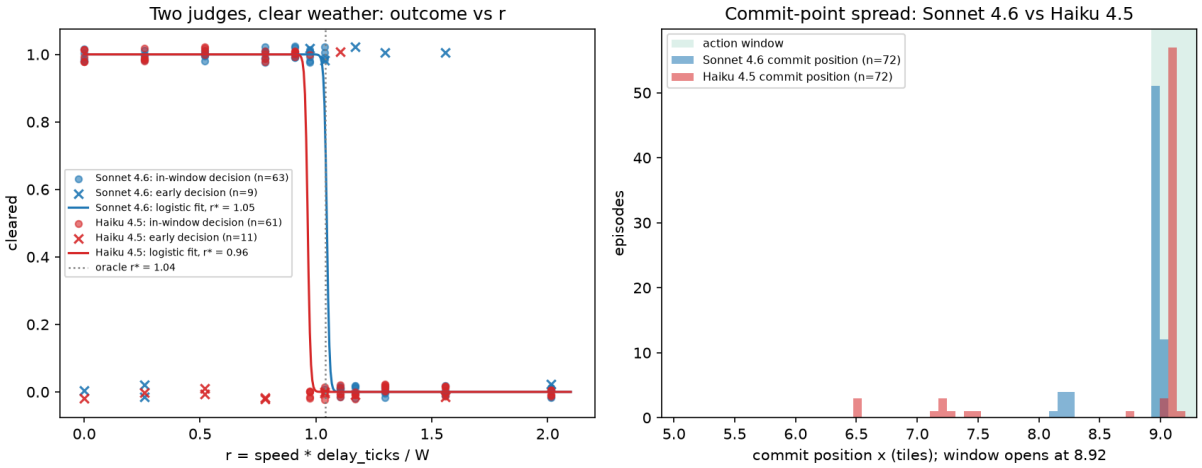



Figure 8: Two judges through the same grid in clear weather: Sonnet 4.6 (blue) and Haiku 4.5 (red). Left: outcome vs. $r$; dots are in-window commits with their logistic fits ($r^\star = 1.05$ and 0.96), crosses early commits. Right: commit-position histograms; Haiku's mode is two ticks later than Sonnet's, which is the whole of the difference in $r^\star$, and its early commits sit two to three tiles before the pit, where no delay can rescue them.

### 4.9 The judge's own latency through the boundary

Every sweep above injects the delay; the judge's real latency enters only through the placement of §4.11. Here we remove the knob. The same judge runs in an emulated real-time loop, and the only thing swept is how fast the world moves while it thinks. Each decision's measured wall-clock latency $L$ is applied as its own delay of round$(L/\tau)$ ticks, where $\tau$ is the clock period (seconds of wall-clock per sim tick), and the judge is not shown another frame until it has answered. That is what a real-time loop running at $\tau$ per tick does, replayed in sim time so the sim stays deterministic and every call's latency is logged (retries are logged too; there were none). We swept $\tau$ from the sim's native 1/60 s, the regime §4.11 places models in, to 4 s per tick, which approaches blocking mode: 12 clock periods × 6 seeds, 72 episodes, 1,948 vision calls, pit width 2, fog 0, the §4.4 judge unchanged (cue prompt, high effort, two-consecutive commit rule).

**Prediction.** A judge sampled every $n$ ticks that commits on its $k$-th consecutive in-window JUMP takes off at $x_1 + k\,v\,n$, where $x_1$, its first in-window sample, lies in $[w_{lo},\ w_{lo} + vn)$. It clears iff that takeoff is at most $x_{hi}$: in $r = vn/W$, iff $r \le 1/k$ at the best sampling phase and $r \le 1/(k+1)$ at the worst. This judge has $k = 2$, so its real-time flip should sit in $r \in [1/3, 1/2]$, not at the injected-delay boundary $r \approx 1$: in real time the window must hold the latency $k$ times plus the phase of the next sample. The injected-delay boundary is the best case.

**The flip lands where the commit rule says.** At the native clock the window is open for 0.3 s of wall-clock, the judge gets one call per episode ($r \approx 11$) and never commits: 0/6. Nothing clears through $\tau = 0.4$ s (**0/30 for** $r \ge 0.45$); the first clears come at $\tau = 0.5$ s, $r = 0.32$, the worst-phase bound 1/3; and once the clock is slow enough, $\tau \ge 1.5$ s ($r \le 0.13$), the same judge clears **18/18**.

Table 1: Clock-rate sweep of the §4.4 judge at its high effort, and of the same judge at low effort. $\tau$ is the wall-clock period of one sim tick; the window is open for $16.3\,\tau$ seconds of wall-clock; $n$ is the median emulated delay in ticks and $r = vn/W$. Six seeds per cell.

| | | high effort (the §4.4 judge) | | | | low effort | | | |
|---|---|---|---|---|---|---|---|---|---|
| $\tau$ (s/tick) | window (s) | calls/ep | $n$ | $r$ | cleared | calls/ep | $n$ | $r$ | cleared |
| 1/60 | 0.3 | 1 | 168 | 10.9 | 0/6 | 2 | 63 | 4.11 | 0/6 |
| 0.1 | 1.6 | 4 | 29 | 1.87 | 0/6 | 11 | 10 | 0.65 | 3/6 |
| 0.2 | 3.3 | 7 | 14 | 0.91 | 0/6 | 20 | 5 | 0.32 | 4/6 |
| 0.3 | 4.9 | 10 | 9 | 0.58 | 0/6 | 27 | 3 | 0.19 | 1/6 |
| 0.4 | 6.5 | 14 | 7 | 0.45 | 0/6 | 36 | 2 | 0.13 | 0/6 |
| 0.5 | 8.2 | 16 | 5 | 0.32 | 2/6 | 40 | 2 | 0.13 | 0/6 |
| 0.6 | 9.8 | 17 | 5 | 0.32 | 3/6 | 47 | 2 | 0.13 | 0/6 |
| 0.8 | 13.0 | 26 | 3 | 0.19 | 5/6 | 71 | 1 | 0.06 | 0/6 |
| 1.0 | 16.3 | 32 | 3 | 0.19 | 3/6 | 78 | 1 | 0.06 | 0/6 |
| 1.5 | 24.5 | 41 | 2 | 0.13 | 6/6 | 78 | 1 | 0.06 | 0/6 |
| 2.0 | 32.6 | 73 | 1 | 0.06 | 6/6 | 79 | 1 | 0.06 | 0/6 |
| 4.0 | 65.2 | 84 | 1 | 0.06 | 6/6 | 80 | 1 | 0.06 | 0/6 |

The injected-delay sweep put this judge's cliff at $r = 1.05$; with its own latency in the loop the cliff is at a third of that, as the arithmetic requires, and the §4.11 placement of "$r \approx 4$–6" for hosted models understates the real-time deficit by the same factor.

**The deliberation stall.** Between those regimes, $\tau = 0.5$ to 1.0 s, the judge clears 13/24, and 10 of the 11 failures are one event. Per-call latency is not a constant of the model (Fig. 9, right). Outside $x \in [7.9, 8.4]$ the 1,878 calls have median 2.6 s and 90th percentile 3.4 s; the 70 calls on frames in $[7.9, 8.4]$ have median 13.6 s, 90th percentile 25.9 s and maximum 32.4 s, and 47 of them exceed 10 s against 12 of 1,878 elsewhere. Those are the frames where the pit's black band approaches, but does not yet touch, the bottom edge — the hardest discrimination the prompt asks for. Three probe calls on the $x = 8.10$ frame returned 273–722 output tokens in 6.6–16.2 s with stop reason `end_turn`, against 107–124 tokens in 2.4–2.9 s at $x = 8.94$: adaptive thinking deliberates on the ambiguous frame and finishes on its own, well short of the token cap. In blocking mode that costs patience. In real time the agent keeps walking: at $\tau = 1$ s the window is 16 s of wall-clock and a 30 s deliberation on the 8.1 frame carries the agent from 8.1 to past the pit's edge before it answers. Ten of the eleven failures between $\tau = 0.5$ and 1.0 s contain such a call ($\geq 10$ s on a frame in $[7.9, 8.4]$), after which the next sample is at $x \geq 9.85$ or in the pit. The judge is not wrong on those episodes — in-window frames drew JUMP on 96/96 calls across the sweep — it is late because it was careful, on exactly the frame before the decision.

The real-time budget is therefore set by the longest latency on the approach, not the median, and the longest one falls on the frame the judge finds hardest, which is the one just before the window opens. A table of median latencies understates a real-time $r$ by the ratio of the deliberation to the median, here about 5×.

**Taking the deliberation away.** If the stall is the price of care, a less careful judge should not stall. We ran the same sweep with the same prompt at low effort (72 episodes, 3,416 calls, Table 1 right, red in Fig. 9). The stall is gone: latency is 1.0 s at the median everywhere, 1.3 s at the

90th percentile, and the maximum on the $[7.9, 8.4]$ frames is 2.5 s. So is the judgment. In-window frames still draw JUMP (18/18), but on the frames between $x = 7.5$ and the window's edge the low-effort judge answers JUMP 54% of the time (233/429; on $x \in [8.0, 8.5)$, 122/128), against 7.6% for the high-effort judge on the same frames. The frames that stall the careful judge are the frames that fool the fast one. With a 54% false-JUMP rate on the approach band and a two-consecutive commit rule, §4.5 says the judge commits early almost surely once it gets more than a handful of looks at that band, and it does: 58/72 episodes commit before the window, every episode at $\tau \geq 0.4$ s, and the sweep clears **8/72**. The two-state chain with the band rate and the band's calls per episode predicts 48 early commits; the residual is clustering within the band (36/36 JUMPs at $x \approx 8.25$), the dependence the first-passage model assumes away. The only clears are at $\tau = 0.1$–$0.3$ s (8/18), where the clock is fast enough that the judge sees the band only two to four times and slow enough that its 1 s latency fits the window ($r = 0.2$–$0.65$): a narrow band where the first-passage term and the timing term, which pull in opposite directions as the clock changes, both happen to be survivable. Effort does not move the judge along the timing axis; it trades the stall for the first-passage failure, and in real time neither judge is viable at the native clock.

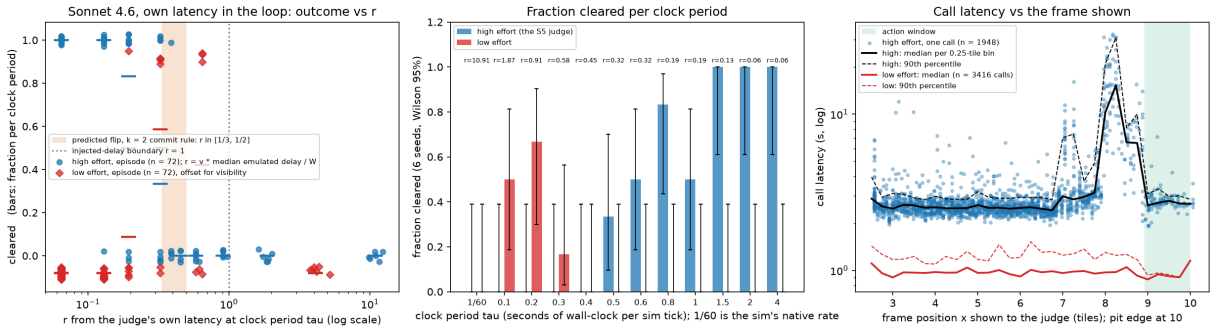



Figure 9: The §4.4 judge with its own latency in the loop. Left: outcome vs. $r$ computed from each episode's emulated delay; the shaded band is the predicted flip for a two-consecutive commit rule, $r \in [1/3, 1/2]$; the dotted line is the injected-delay boundary. Middle: fraction cleared per clock period (Wilson 95%); the native clock is the leftmost bar. Right: every call's latency against the frame it was shown; the high-effort judge answers in 2.6 s everywhere except the frames just before the window, where it deliberates for 10–32 s and, in real time, walks into the pit while doing so. Red: the same judge at low effort, which never stalls (1.0 s flat) and instead answers JUMP on 54% of the frames in that band, so it commits early in 58/72 episodes and clears 8/72.

## 4.10 Perception axis

Does a vision model judge this jump from a single frame? We measured it through the API, blind, on a balanced battery. For each fog level we render 100 standing frames at pit width 2 — 50 at distinct out-of-window positions ($x = 6.00$–$8.85$) and 50 inside the window ($x = 8.93$–$9.99$) — shuffle them into neutral names with a fixed seed, and show each alone (no coordinates, window, or labels) to three hosted models and one local open model, which answer JUMP or WAIT; each answer is scored against the sim's `in_window` label, so chance on this set is 50%. We ran two prompts. The *neutral* prompt describes the task (auto-walking toward a pit you must jump, too early lands short, too late walks in) and says nothing about what the pit looks like or where in the frame to look. The *cue* prompt is the §4.4 judge's: it tells the model to read the strip touching the bottom border of the frame and answer tiles or black. Sonnet 4.6 and Opus 4.8 run adaptive thinking at medium effort; Haiku 4.5 predates adaptive thinking and runs with a 2k-token thinking

budget; Qwen2.5-VL 7B runs locally through Ollama with no thinking. 1,600 calls in all (Table 2, Fig. 10).

Table 2: Blind single-frame judgment accuracy through the API, $n = 100$ frames per cell (50 in-window, 50 out). Wilson 95% intervals are ±5–10 points at the extremes and ±10 at chance.

| | Neutral prompt | | Cue prompt | |
|---|---|---|---|---|
| Model | Clear | Fog | Clear | Fog |
| Haiku 4.5 | 52% | 48% | 97% | 68% |
| Sonnet 4.6 | 50% | 50% | 91% | 93% |
| Opus 4.8 | 49% | 50% | 99% | 94% |
| Qwen2.5-VL 7B (local) | 50% | 50% | 50% | 50% |

Four things fall out. **(1) Unprompted, no model judges the cue at all.** With the neutral prompt every model is at chance at both fog levels; Sonnet and Opus answered WAIT on 199 of 200 frames each without engaging extended thinking, Qwen on all 200, and Haiku's JUMPs are scattered across the approach. **(2) Told where to look, every model judges it, including the smallest.** The cue prompt lifts clear-weather accuracy to 97% (Haiku), 91% (Sonnet) and 99% (Opus); each contrast with the neutral prompt is Fisher exact $p < 10^{-9}$. **(3) Fog degrades the cued judgment where the model is weakest, not where it is strongest.** Haiku falls from 97% to 68% ($p = 3.5 \times 10^{-8}$), missing 30 of 50 in-window frames; Opus from 99% to 94% ($p = 0.12$); Sonnet is unchanged (91% vs. 93%, $p = 0.8$), its clear-weather errors being false JUMPs in the tile before the window ($x = 7.98$–$8.73$, 9/50 at medium effort) rather than misses. The capability ordering is visible only under fog and only with the cue. **(4) The local 7B model never judges it.** Qwen2.5-VL answered WAIT on every neutral frame and JUMP on every cue frame, clear or fog; under the cue prompt it echoed the prompt's first line verbatim instead of answering it, then emitted JUMP. Both arms are a constant answer, so both sit at exactly 50% on the balanced set. A model that follows the cue instruction (the hosted three) and one that cannot (Qwen) are indistinguishable with the neutral prompt and 49 points apart with the cue.

**Retraction of the pilot result.** An earlier draft reported, from a 10-frame-per-cell battery judged by Claude Code subagents rather than API calls, that Opus and Sonnet read the depth cue unprompted (90% and 80%) and collapsed to chance under fog, and called the collapse a signature of genuine depth reading. That result does not replicate under the controlled protocol above: unprompted, the same models are at chance in both conditions. We keep the pilot file in the repository for the record (`perception_judgments.jsonl`) and withdraw the claim; the subagent path evidently supplied context that a single-frame API call does not.

This reframes the perception half of the dissociation. "Perception" of this scene is not a fixed property of the model: it is almost entirely whether the prompt names the cue, and once it does, capability shows up as the robustness of that reading to fog. Both measurements are at zero delay and independent of the timing axis. The §4.4 judge is the cue-prompted Sonnet at high effort, so its per-frame accuracy at fog 0 is the 91% cell above with the effort raised; §4.5 shows why a 9/50 pre-window false-JUMP rate at medium effort would be fatal in closed loop, and §4.7 is that judge taken through the boundary under fog.

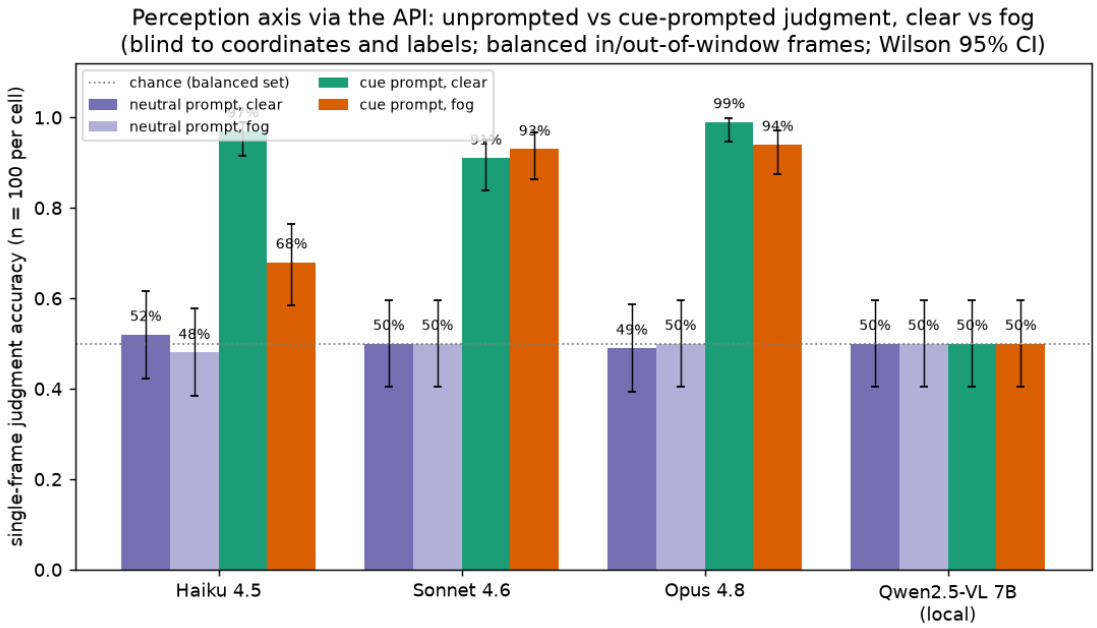



Figure 10: Perception axis through the API. Neutral prompt: every model at chance, clear or fog. Cue prompt: every hosted model near ceiling in clear weather; fog costs Haiku 29 points, Opus 5, Sonnet none. The local Qwen2.5-VL 7B gives a constant answer in both arms (WAIT, then JUMP) and stays at chance. Wilson 95% intervals.

### 4.11 Placing real models' latency on the axis

$r = L/T_{\text{window}}$ places a measured decision latency on the same axis. At pit width 2 and $v = 0.07$ tiles/tick the window is open for $T_{\text{window}} = (W/v) \cdot \text{tick} = 257\,\text{ms}$ (tick $= 1/60\,\text{s}$), so the calibrated boundary corresponds to $r^\star \cdot T_{\text{window}} \approx 270\,\text{ms}$. We timed five vision models on the same 20-frame battery, in each model's fastest single-call configuration (Table 3, Fig. 11): hosted frontier models answer in 1–1.5 s ($r \approx 4$–$6$) and local open VLMs in 8–12 s ($r \approx 31$–$47$). None is inside the window at this speed. The multiples are a property of the speed we chose — $T_{\text{window}}$ scales as $1/v$, and a world slowed 4× puts the hosted models at the boundary — which is the point: $r$ is the quantity to report, and a deployment moves it by pausing or slowing the world as readily as by cutting latency. Two caveats on the measurement. These are time-to-answer numbers, not judgments: in the fast configuration the hosted models answered WAIT on nearly every frame (Haiku and Sonnet 20/20, Opus 18/20; the unprompted default of §4.10) and the local models JUMP on nearly every frame; a configuration that discriminates (the §4.4 judge, ∼5 s/call) is slower, so these $r$ values are lower bounds. And each local model's first call is an order of magnitude faster than the rest (0.4–0.6 s against 8–12 s), consistent with a warm cache; the medians are unaffected.

## 5 Discussion

The two failure axes have opposite fixes, and the framing makes the requirement quantitative. A perception failure wants a better model: no amount of extra time helps a judge that reports no distinguishable pit band under fog. A timing failure wants a slower clock: a judge that commits a fraction $\delta$ of the window past its leading edge is viable exactly when $r = L/T_{\text{window}} < 1 - \delta$ (plus

Table 3: Measured per-model decision latency (median of $n = 20$ single-frame calls, fastest configuration) against the 257 ms window at pit width 2.

| Model | Median $L$ | $r = L/\text{window}$ |
|---|---|---|
| Haiku 4.5 (hosted API) | 0.95 s | $\approx 4$ |
| Sonnet 4.6 (hosted API) | 1.45 s | $\approx 6$ |
| Opus 4.8 (hosted API) | 1.53 s | $\approx 6$ |
| Qwen2.5-VL 7B (local) | 7.9 s | $\approx 31$ |
| llama3.2-vision 11B (local) | 12.1 s | $\approx 47$ |

the same-tick tolerance). For the oracle and for the Sonnet judge of §4.4, $\delta \approx 0.02$, which is why both sit at $r^\star \approx 1$; a judge that committed mid-window would have $r^\star = 0.5$ in the same apparatus, and the spread of its commit point (§4.6) would turn the step into a curve. A deployment can cross the boundary by pausing the world ($r \to 0$), slowing it (raising $T_{\text{window}}$), or cutting latency — and at this sim speed the measured field needs 4–47× on the latter two. §4.4 is the direct demonstration: the same model, same judgment, clears 32/32 below the boundary and 0/26 above it, with nothing varied but $r$. The practical reading is that pausable or turn-based deployment converts every slow-but-accurate model we measured from failing to viable, with no change to the model; conversely, a real-time benchmark score is a product of judgment *and* time regime, and "VLMs are too slow to play games" is partly a fact about the clock they are handed. The delay-rescued misjudgments add a sharper corollary: past the boundary, scores can even *reward* bad perception — the axes compensate, and an aggregate benchmark cannot tell. The fog sweep (§4.7) makes the compensation quantitative: the same aggregate score, 41/72, is 36 correct plus 5 rescued in clear weather and 19 plus 22 under fog, and pooling the fog episodes yields a smooth psychometric curve that belongs to the mixture, not to the judge. And the first-passage result (§4.5) says that even below the boundary, a per-frame judge has to be evaluated per episode: a false-positive rate that rounds to zero per frame does not round to zero per approach.

# 6 Limitations

- **The real-model sweeps (§4.4, §4.7, §4.8) are two models at one pit width, at two fog levels.** Both judges use a revised prompt (the latency-battery prompt elicits no JUMPs near the window from any hosted model) plus a two-consecutive-JUMP commit rule, developed against and validated on the same 108 approach frames the sweep draws from — a judge tuned to this scene, and a documented apparatus change, not the shipped default. Their $r$ axis is engineered by injected delay with the model's judgment in the loop; §4.9 puts the judge's own latency in the loop, but in an emulated loop and for Sonnet only. The anticipated *noisy* psychometric curve did not materialize in either sweep: fog spread the commit point, but mostly *before* the window, and outcome is a deterministic function of commit position and delay, so the in-window transition stayed a step. A smooth in-window curve needs a judge whose commit varies within the window.
- **The location $r \approx 1$ is definitional for leading-edge policies.** The agreement between the pit and aiming oracles (§4.1, §4.3) checks the apparatus; it is not evidence that real agents on other tasks share a boundary. Whether real judges on richer, multi-decision tasks have a stable commit point — the quantity that would make $r$ predictive for them — is untested; §4.6 shows one judge on one scene does.

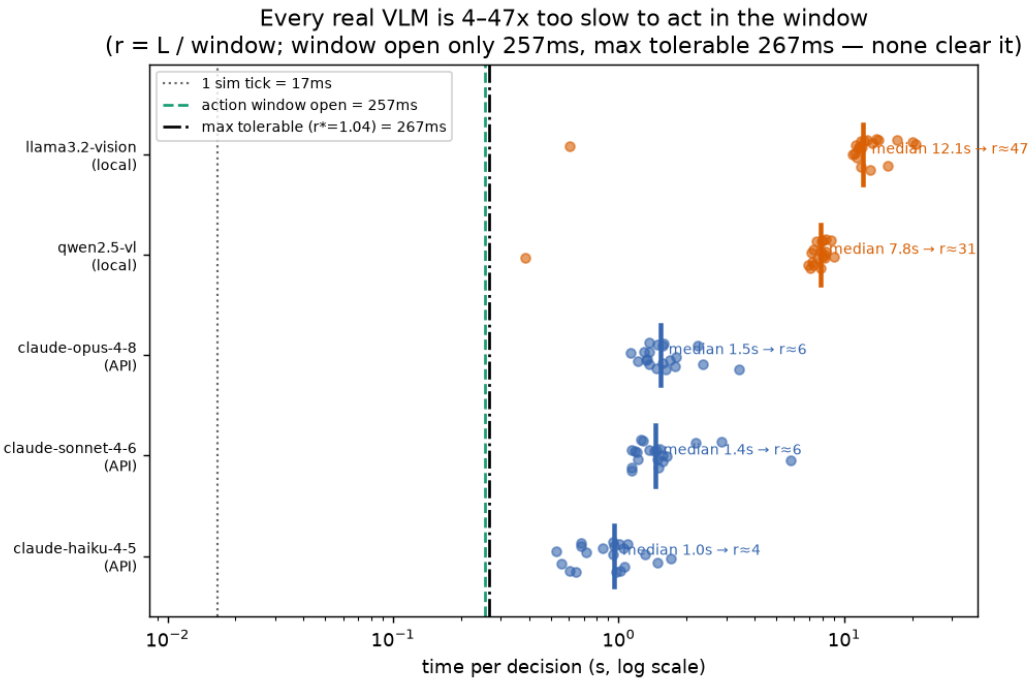



Figure 11: Real per-model decision latency vs. the time budget at this sim speed. Hosted Haiku/Sonnet/Opus ($r \approx 4$–$6$) and local Qwen/llama3.2-vision ($r \approx 31/47$); all past the ≈270 ms budget.

- **The first-passage curves assume independent per-frame errors.** The final judge's residual errors cluster in the tile before the window, so the curves in Fig. 6 are an idealization of the mechanism, and the 1–2% per-frame rates they are evaluated at come from earlier prompt versions, not from the final judge (whose far-field rate on the battery was zero in 237 samples).
- **The perception axis is one scene, two prompts, three hosted models at medium effort and one local 7B model.** The cue prompt is the §4.4 judge's, so the cue-arm numbers are that judge's single-frame accuracy at a lower effort than the sweep used; a prompt that named a different cue, or a scene without a pixel-sharp cue, could land anywhere between the two arms. Judges are blind to coordinates and labels but not to the task. The local model's chance result is an instruction-following failure (a constant answer), not a measurement of its depth perception.
- **The real-time loop of §4.9 is emulated.** Each call's measured latency is replayed as ticks of delay in a deterministic sim; there is no render jitter, no dropped frame, and the sim never lags. It is the real latency in the loop, but a real-time implementation would be at least this bad, not better.
- **The latency battery measures time-to-answer, not judgment, at one arbitrary sim speed.** In the fast configuration the hosted models answered WAIT on nearly every frame; a configuration that discriminates is slower, so the measured $r$ values are lower bounds. The "4–47×" multiples scale with $1/v$ and would halve at half the approach speed; $r$ is the portable number. Latencies are network- and hardware-dependent, but they are real decision latencies,

not stand-ins.

- **No human baseline** — the oracle stands in as the perfect-judgment ceiling. Human psychophysics is future work.

## 7 Future work

Extending the real-model sweeps to more models and pit widths; a cue without a sharp onset, to spread the commit point inside the window rather than before it (the quantity that would turn the step into a psychometric curve); a multi-provider latency panel (Gemini/GPT + more open VLMs); a human psychophysics arm; and multi-decision tasks, to test whether real judges have a stable enough commit point for $r$ to stay predictive beyond single reflexes.

## Reproducibility

Code, data, and figure-generating scripts are at `https://github.com/star7js/judge-vs-react`, under `eval/pit/` (analysis and sweeps) and `raycaster/` (the deterministic sim, including `--task aim` for the second task). The perception judgments (`perception_judgments.jsonl`, 80 rows across 4 models), per-decision latencies (`vlm_latency.jsonl`, 100 rows across 5 models), the oracle sweeps (default grid and the 1-tick `oracle_edge_sweep.jsonl`), the aim sweep, and the real-model sweep (`eval/pit/realmodel/`: driver, revised judge, 108-frame battery validation, the 72-episode JSONL, and the fog-1 battery and 72-episode fog sweep) and the API perception battery (`run_perception_api.py`, `perception_judgments_v2.jsonl`) are committed. The local-VLM paths (Ollama) and the Claude subagent judges run without an API key; the hosted-API latency numbers and the real-model sweep required one.

## A Control: action-repeat does not rescue

Consulting the policy only every $k$ ticks and holding its last decision is a common cost lever. It adds up to $k-1$ ticks of polling delay on top of the explicit $D$, so it must shift the transition to *lower* explicit $r$, and it does. Measured $r^\star$ vs. $k$ at pit width 2 ($D \in \{0, 2, \ldots, 40\}$, one seed): $k$=1 $\to$ 1.10, $k$=2, 4 $\to$ 0.97, $k$=8, 16 $\to$ 0.71; the ties are grid quantization (a 2-tick delay step is 0.13 in $r$ at this width). This is a control, confirming that the apparatus charges polling delay the same as injected delay. Action-repeat is a compute/cost lever, not a latency fix.

## References


[1] M. Lin et al. *OmniGameArena: A Unified UE5 Benchmark for VLM Game Agents with Improvement Dynamics*. arXiv:2606.09826, 2026.

[2] A. L. Zhang, T. L. Griffiths, K. Narasimhan, O. Press. *VideoGameBench: Can Vision-Language Models complete popular video games?* arXiv:2505.18134, 2025.

[3] H. Kang et al. *Win Fast or Lose Slow: Balancing Speed and Accuracy in Latency-Sensitive Decisions of LLMs*. NeurIPS 2025, arXiv:2505.19481.

[4] X. Fu et al. *BLINK: Multimodal Large Language Models Can See but Not Perceive*. ECCV 2024, arXiv:2404.12390.

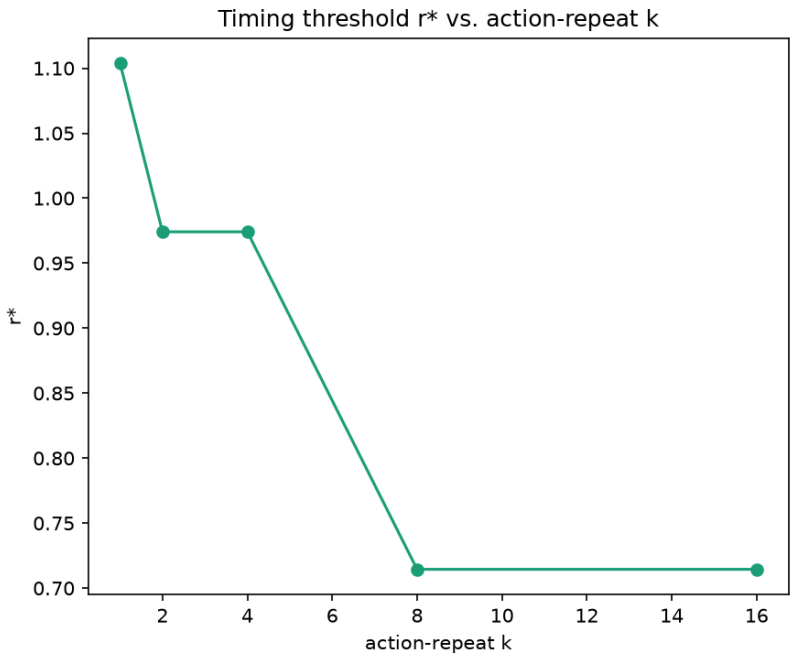



Figure 12: Action-repeat does not rescue the transition; larger $k$ fails at lower explicit $r$.

[5] S. Ramstedt, C. Pal. *Real-Time Reinforcement Learning*. NeurIPS 2019, arXiv:1911.04448.

[6] V. Firoiu, T. Ju, J. Tenenbaum. *At Human Speed: Deep Reinforcement Learning with Action Delay*. arXiv:1810.07286, 2018.

[7] N. K. R. Sehgal, S. C. Guntuku, L. Ungar. *Real-Time Deadlines Reveal Temporal Awareness Failures in LLM Strategic Dialogues*. arXiv:2601.13206, 2026.